

ADVERSARIAL SKILL TUNING: SELF-PLAY BETWEEN NATURAL-LANGUAGE POLICIES FOR MATHEMATICAL ERROR DETECTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Adversarial self-play is an attractive recipe for training verifiers: a perturber corrupts correct reasoning, a verifier hunts for the corruption, and neither needs human error annotation. In practice the recipe is brittle — the game is difficult to balance, and reward hacking is endemic. We ask whether the same game can be played at the level of *natural-language policies* rather than model weights. We instantiate a perturber and a verifier as *agent skills*: versioned prose documents that a coding agent reads and follows. After each round of self-play the agent inspects the scored episodes and rewrites the losing policy, producing `perturb-v{N+1}` and `verify-v{N+1}`, in direct analogy to a policy-gradient step but with the update expressed in text. Rewards are computed by the same programmatic reward function an RL implementation would use, so the two regimes optimise an identical objective. We separate each skill into an *invariant* contract that the updater may not touch and a *tuned* policy segment that it rewrites, and we require the updater to file reward-earning behaviours that exploit the scorer as bugs rather than lessons. Across three rounds on Hendrycks MATH, self-play metrics move as expected, and we evaluate every verifier version on held-out ProcessBench, which the tuning loop is structurally prevented from seeing. Transfer is real but not monotone: the round-2 policy improves the ProcessBench F1 from 0.869 to 0.897, while the round-3 policy regresses to 0.874 — an over-fit to the current perturber that the self-play reward alone reported as a gain. We argue that a mandatory external benchmark is not an optional add-on to self-play but the only signal that distinguishes learning to verify from learning one’s opponent.

1 INTRODUCTION

Mathematical and statistical errors survive peer review at a rate that has motivated a small industry of automated checkers. Recent reporting describes volunteer and commercial projects applying large language models to published literature at scale, uncovering arithmetic and methodological mistakes that human referees missed, while cautioning that the tools themselves produce unverified flags (Gibney, 2025). The underlying capability — reading a piece of reasoning and locating the step where it first goes wrong — is also the capability that scalable oversight of language models requires (Amodei et al., 2016; Bowman et al., 2022; Saunders et al., 2022), and it is measured directly by ProcessBench (Zheng et al., 2024).

Training such a verifier is bottlenecked by supervision: annotated process errors are expensive, and existing corpora are small relative to the diversity of ways a derivation can fail. Adversarial self-play offers an escape. A *perturber* takes a correct solution and injects errors, producing ground truth for free; a *verifier* tries to recover them; the perturber is rewarded when the verifier fails and vice versa. The structure is that of a generative adversarial network (Goodfellow et al., 2014) transposed onto discrete text, and of self-play in games (Silver et al., 2017).

Our own experience implementing this loop with reinforcement learning — GRPO (Shao et al., 2024) for the perturber, DPO (Rafailov et al., 2023) for the verifier — was that the machinery works and the *game* does not. Across three training rounds the perturber found three successive reward hacks: declaring errors it had not injected, injecting a single mistake and declaring it k times,

and finally producing text that was redundant rather than wrong. Each was patched in the reward function, at the cost of a round of GPU time to discover it. This is the standard reward-hacking pathology (Skalse et al., 2022): the agent optimises the measurement, not the task.

This paper asks whether the same adversarial game becomes easier to steer when the policies are *prose*. We implement the perturber and verifier as *agent skills* — markdown documents that a coding agent (Anthropic, 2025) loads and follows as its policy for the round. After each round, a static *updater* skill reads the scored episodes and rewrites the policy text. The artefact being optimised is human-readable at every step: one can see the policy that produced a reward, and see the edit that a round of evidence motivated. Our contributions are:

- **A formalisation of adversarial skill tuning** (§3) as alternating coordinate ascent over a pair of natural-language policies, with the reward computed by exactly the programmatic function an RL implementation would use.
- **Three structural guardrails** that make the loop auditable: an invariant/tuned split within each skill, mechanically enforced; a *faithfulness filter* requiring the updater to file scorer-exploiting wins as bugs rather than convert them to policy; and enforced context isolation between the two players.
- **An external-validity protocol** on ProcessBench that the tuning loop cannot read, together with initial results over three rounds (§5) showing that self-play gains transfer in round 2 and fail to transfer in round 3.

2 RELATED WORK

Process supervision and error identification. Outcome-supervised verifiers (Cobbe et al., 2021) were superseded by process supervision, where each reasoning step carries a label (Lightman et al., 2023), and by process reward models trained with automatically synthesised step labels (Wang et al., 2024). ProcessBench (Zheng et al., 2024) evaluates the resulting critics directly: given a problem and a step-segmented solution, a model must return the index of the earliest erroneous step or declare the chain correct. Its 3,400 human-annotated cases span four difficulty tiers built from GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), OlympiadBench (He et al., 2024) and Omni-MATH (Gao et al., 2024). We adopt both its task definition and its metric.

Automated error detection in scientific writing. The applied motivation for verifier research is documented by Gibney (2025), who describes projects screening the published literature with language models — one approaching authors privately after analysing several hundred papers, another publicly flagging candidate errors across tens of thousands — and the attendant risk of unconfirmed flags. That risk is precisely a precision problem, and it is why our reward and our benchmark metric both penalise false alarms as heavily as misses.

Self-play and self-improvement. Self-play removes the annotation bottleneck by having a model generate its own curriculum (Silver et al., 2017; Chen et al., 2024), and self-rewarding schemes (Yuan et al., 2024) and self-refinement (Madaan et al., 2023) let a model supply its own feedback. Our setting differs in that the reward is not model-judged: it is a deterministic program over the perturber’s declared ground truth, which is what makes the perturber’s incentive to cheat concrete and detectable.

Optimising prompts and skills instead of weights. A line of work treats the prompt as the parameter, optimising it by search (Zhou et al., 2023), by natural-language “gradients” (Pryzant et al., 2023), by using the model as an optimiser over its own instructions (Yang et al., 2024), or by compiling declarative pipelines (Khatab et al., 2024). Closest to us is Ctx2Skill (Si et al., 2026), which extracts natural-language *skills* from long contexts through multi-agent self-play: a Challenger poses probing tasks, a Reasoner answers, a Judge scores, and Proposer/Generator agents synthesise skill updates from failures, with a *Cross-Time Replay* mechanism that re-evaluates historical skill candidates against curated probe sets to prevent adversarial collapse.

We share the premise — that a natural-language skill is a trainable object, and that adversarial interaction can produce one without human annotation — and differ in three ways. First, our reward

is a programmatic function of a machine-checkable ground truth rather than a model judge, so credit assignment is exact and the failure mode we must guard against is scorer exploitation rather than judge bias. Second, both of our players are skills and both are updated, so the curriculum and the solver co-evolve. Third, where Ctx2Skill guards against collapse by replaying historical candidates on curated in-domain probes, we guard against it with an external human-annotated benchmark that the update mechanism is structurally forbidden to read; as §5 shows, that distinction is load-bearing, because the round in which our loop over-fits is invisible from inside the game.

3 METHOD

3.1 THE ADVERSARIAL EPISODE

Let $\mathcal{D}$ be a corpus of solved problems, and let $(p, s) \sim \mathcal{D}$ be a problem with a correct step-by-step solution. Fix a budget $k \in \mathbb{N}$ of errors per episode. A *policy* in our setting is a natural-language document σ ; writing $\pi(\cdot \mid \cdot; \sigma)$ for the behaviour induced when an agent follows σ , one episode is

$$(\hat{s}, E) \sim \pi_P(\cdot \mid p, s, k; \sigma_P), \quad E = \{e_i\}_{i=1}^k, \quad e_i = (o_i, \iota_i, \tau_i, \rho_i), \quad (1)$$

$$C \sim \pi_V(\cdot \mid p, \hat{s}; \sigma_V), \quad C = \{c_j\}_{j=1}^m, \quad c_j = (q_j, x_j, \varsigma_j). \quad (2)$$

The perturber returns a corrupted solution $\hat{s}$ together with a machine-readable log E in which each error names the original span o_i , the injected span ι_i , a type τ_i drawn from a fixed 26-way taxonomy of student misconceptions (Rittle-Johnson et al., 2025), and a rationale ρ_i . The verifier sees only p and $\hat{s}$ — never E — and returns claims c_j consisting of a quoted span q_j , an explanation x_j and an optional type ς_j .

3.2 MATCHING, ERROR UNITS, AND REWARD

Scoring requires aligning claims to injected errors. Let $\text{IoU}(\cdot, \cdot)$ be character-level intersection-over-union after normalisation (LaTeX escaping, math delimiters, alignment characters, whitespace). Define the alignment score

$$\mu(e, c) = \max \left\{ \underbrace{\text{IoU}(\iota_e, q_c)}_{\text{span overlap}}, \underbrace{\gamma \cdot \mathbb{1}[q_c \succ \delta(o_e, \iota_e)]}_{\text{diff coverage}}, \underbrace{\lambda(\iota_e, q_c)}_{\text{containment}} \right\}, \quad (3)$$

where $\delta(o_e, \iota_e)$ is the set of fragments a character diff marks as changed, $q_c \succ \delta$ holds when the quote is anchored inside the error span *and* contains a changed fragment, $\gamma = 0.9$, and λ is the length ratio of the shorter normalised string when one contains the other. The middle term matters: it distinguishes a claim that quotes the actual mistake from one that quotes an untouched clause of the same sentence.

A greedy one-to-one assignment $M \subseteq E \times C$ then takes, for each error in turn, the highest-scoring unused claim above a threshold $\theta = 0.5$.

Error units. A perturber can cap the verifier’s recall at $1/k$ by injecting one mistake and declaring it k times — as a root error plus its propagated consequences, as overlapping rewrites of one region, or as pure restatement. We therefore quotient E before computing recall. Let $\sim$ be the reflexive-symmetric-transitive closure of the relation that holds between two errors when they occupy overlapping or containing spans, share a changed diff fragment, differ by a token-Jaccard below 0.6, or when one modifies a `\boxed{\cdot}` answer downstream of the other. The *error units* are the quotient $\mathcal{U} = E/\sim$, and we restrict to units actually present in $\hat{s}$:

$$\mathcal{U}^{\text{eff}} = \{U \in \mathcal{U} : \exists e \in U, \iota_e \sqsubseteq \hat{s}\}, \quad k_{\text{eff}} = |\{e \in E : \iota_e \sqsubseteq \hat{s}\}|, \quad (4)$$

with $\sqsubseteq$ denoting normalised containment. Recall is measured over units and precision over claims:

$$\text{rec} = \frac{|\{U \in \mathcal{U}^{\text{eff}} : \exists e \in U, (e, \cdot) \in M\}|}{|\mathcal{U}^{\text{eff}}|}, \quad \text{prec} = \frac{|M|}{|C|}. \quad (5)$$

Quotienting only the recall denominator is deliberate: a verifier that flags both a root error and its propagation keeps full precision, while a perturber that stacks gains nothing.

Rewards. With F_β the usual harmonic combination of Eqs. (5),

$$r_V = F_\beta(\text{prec}, \text{rec}) - \lambda_{\text{spam}} \max(0, |C| - \varrho \max(1, k_{\text{eff}})) - \lambda_{\text{rep}} \mathbb{1}[\text{output collapsed}], \quad (6)$$

$$r_P = (1 - \text{rec}) - \lambda_{\text{miss}}(k - k_{\text{eff}}) - \lambda_{\text{dup}} d(E) - \lambda_{\text{hist}} d(E, E^{<t}), \quad (7)$$

where $d(E)$ counts near-verbatim repeats within the episode and $d(E, E^{<t})$ counts injections that are ≥ 0.85 similar to one used in an earlier round. Both are overridden by a dominating format penalty when the perturber’s output is not a valid instance of the schema:

$$r_P = \begin{cases} \phi_{\text{json}} = -10, & \text{output is not parseable JSON,} \\ \phi_{\text{schema}} = -5, & \text{parses but violates the contract,} \\ \text{Eq. (7),} & \text{otherwise.} \end{cases} \quad (8)$$

The two-level penalty exists because a uniform penalty makes every failing episode score identically, which in the RL implementation eliminated the within-group reward variance that GRPO learns from; it plays the same role here by giving the updater a gradient across failure severities.

3.3 SKILL TUNING AS A TEXTUAL POLICY UPDATE

Each policy document is partitioned into two segments,

$$\sigma = \left(\underbrace{\iota}_{\text{invariant}}, \underbrace{\varpi}_{\text{tuned}} \right), \quad (9)$$

where ι transcribes the I/O contract, the taxonomy and the reward definition from the implementation, and ϖ holds the strategy. A round t produces a scored dataset

$$\mathcal{E}^t = \left\{ (p_n, s_n, \hat{s}_n, E_n, C_n, M_n, r_n^P, r_n^V) \right\}_{n=1}^N. \quad (10)$$

The update operator Λ is a language model that reads $\mathcal{E}^t$ and the current policy and emits a new one:

$$\varpi^{t+1} = \Lambda(\varpi^t, \mathcal{A}(\mathcal{E}^t)), \quad \iota^{t+1} = \iota^t, \quad (11)$$

subject to $|\text{edits}| \leq 5$, each edit citing at least one episode index. Equation (11) is the textual analogue of a policy-gradient step: $\mathcal{E}^t$ plays the role of the sampled batch, the updater’s diagnosis plays the role of credit assignment, and the edit budget plays the role of a trust region. Alternating the two updates,

$$\varpi_P^{t+1} = \Lambda_P(\varpi_P^t, \mathcal{A}(\mathcal{E}^t)), \quad \varpi_V^{t+1} = \Lambda_V(\varpi_V^t, \mathcal{A}(\mathcal{E}^t)), \quad (12)$$

is coordinate ascent on the pair of objectives $J_P(\sigma_P, \sigma_V) = \mathbb{E}[r_P]$ and $J_V(\sigma_P, \sigma_V) = \mathbb{E}[r_V]$ of the underlying min-max game. A `freeze` flag disables either updater, recovering the ablation in which one player is held fixed.

The faithfulness filter. The operator $\mathcal{A}$ in Eq. (11) is the crux. Let $X^t \subseteq \mathcal{E}^t$ be the episodes whose reward was earned by exploiting the scorer rather than the task — for the perturber, stacking, restatement, or burying a small change inside a long declared span; for the verifier, quoting broadly to maximise overlap, shotgunning claims, or keying on the opponent’s habits. Then

$$\mathcal{A}(\mathcal{E}^t) = \mathcal{E}^t \setminus X^t, \quad (13)$$

and X^t is written to a report as a set of candidate defects in the reward implementation. Without $\mathcal{A}$, skill tuning would rediscover in prose exactly the hacks that RL found in weights, and would do so faster, because a language model is better at reading a reward function than a gradient is. The test the updater is instructed to apply is not “did this score well” but “would a competent mathematician call this a real mistake, and a hard one”.

Enforced isolation. Because a single agent plays both roles, leakage is the default failure. The harness makes it structural: after the perturber writes its output, a program validates it and emits a file containing only $(p, \hat{s})$; the verifier pass runs in a fresh agent context scoped to a directory holding nothing else. A round in which the isolation cannot be established is discarded rather than scored.

Convergence. The loop halts when the round budget is exhausted, or when every active updater reports that $\mathcal{A}(\mathcal{E}^t)$ contains no admissible edit, or when the round-over-round change in the relevant metric stays inside a band ($|\Delta r_P| < 0.05$, $|\Delta F_1^{\text{self-play}}| < 0.05$) for two consecutive rounds.

4 EXPERIMENTAL SETUP

Implementation. Both players and both updaters are skills in the Claude Code agent harness (Anthropic, 2025), executed by Claude Opus 5. Episode sampling, perturbation validation, reward computation and aggregation are a Python harness that shares its reward module with the repository’s RL trainers, so a skill-tuning episode and an RL episode are scored by identical code.

Self-play rounds. Each round draws $N = 8$ problems from the Hendrycks MATH training split (Hendrycks et al., 2021) with $k = 3$ errors per episode, sampling without reuse across rounds. Neither player is frozen. We report three completed rounds; a fourth is in progress and is not analysed here.

External evaluation. After each round the newly produced verifier is evaluated on ProcessBench (Zheng et al., 2024). We sample 20 items per subset, balanced between erroneous and correct chains, fixed by a seed that does not depend on the round, so all versions are scored on the same 80 held-out items. Each item is presented as a single text with explicit boundaries,

$$\langle \text{step}_0 \rangle s_0 \langle / \text{step}_0 \rangle \cdots \langle \text{step}_{n-1} \rangle s_{n-1} \langle / \text{step}_{n-1} \rangle,$$

so the whole chain is judged in context while every claim remains addressable to a step. The verifier’s claim set is reduced to a ProcessBench prediction by

$$\hat{y} = \begin{cases} -1, & C = \emptyset, \\ \min_{c \in C} \text{step}(c), & \text{otherwise,} \end{cases} \quad (14)$$

where $\text{step}(c)$ is the claim’s declared index, or the index of the block containing its quoted span when the declaration is missing or out of range. A claim set that resolves to no step is scored wrong against both classes: accusing something unlocatable is not a detection. Following Zheng et al. (2024) we report the accuracies on erroneous and correct chains and their harmonic mean,

$$\text{Acc}_{\text{err}} = \frac{1}{|\mathcal{I}_{-}|} \sum_{i \in \mathcal{I}_{-}} \mathbb{1}[\hat{y}_i = y_i], \quad \text{Acc}_{\text{cor}} = \frac{1}{|\mathcal{I}_{+}|} \sum_{i \in \mathcal{I}_{+}} \mathbb{1}[\hat{y}_i = -1], \quad F_1 = \frac{2 \text{Acc}_{\text{err}} \text{Acc}_{\text{cor}}}{\text{Acc}_{\text{err}} + \text{Acc}_{\text{cor}}}. \quad (15)$$

The harmonic mean is essential rather than cosmetic: a verifier that never claims anything attains $\text{Acc}_{\text{cor}} = 1$ and a balanced accuracy of 0.5, but $F_1 = 0$.

Leakage control. ProcessBench is a test set for this loop and is never read by it. Items and gold labels live in separate directories; the evaluation runs in a fresh agent scoped to the items; the summary artefact contains identifiers, hit/miss flags and aggregates but no item text and no labels; and both updaters are explicitly forbidden from reading the evaluation tree. No ProcessBench content enters a skill, a rollout log, or the orchestrating context at any point.

5 RESULTS

5.1 SELF-PLAY DYNAMICS

Table 1 reports the three completed rounds. Format validity is perfect throughout — the contract segment of the perturber skill states the schema constraints explicitly, and unlike the RL perturber, which failed the schema in 71% of round-1 episodes, the skill perturber never violates it. The adversarial pressure is visible in the trend: mean r_P rises from 0.125 to 0.313 while verifier recall falls from 0.875 to 0.688, i.e. the perturber is winning ground over the three rounds.

The qualitative record is more informative than the aggregates. Round 1 produced five perturber edits and five verifier edits, all traceable to specific episodes: the verifier’s anti-propagation rule had

Table 1: Self-play rounds. Each round is 8 episodes with $k = 3$ on Hendrycks MATH; both players update after every round. Recall is over error units, Eq. (5). FP counts verifier claims matching no injected error.

Round	Perturber	Verifier	Fmt. valid	r_P	r_V	Recall	Prec.	Units/ep	FP
1	v1	v1	8/8	0.125	0.900	0.875	0.938	2.75	1
2	v2	v2	8/8	0.167	0.833	0.833	0.833	3.00	4
3	v3	v3	8/8	0.313	0.750	0.688	0.875	2.88	2

Table 2: ProcessBench, all three tuned verifier versions, $n = 80$ (20 per subset, balanced erroneous/correct, identical items across versions). F_1 is Eq. (15); “Pooled” computes it over all 80 items, “Macro” averages the four subset F_1 values as in Zheng et al. (2024).

Verifier	GSM8K	MATH	Olympiad.	Omni-MATH	Macro	Pooled
verify-v2 (after R1)	0.889	0.889	0.889	0.800	0.867	0.869
verify-v3 (after R2)	0.889	0.947	0.947	0.800	0.896	0.897
verify-v4 (after R3)	0.889	0.947	0.847	0.800	0.871	0.874

been over-applied, skipping two independently invalid lines because they *looked* like consequences of an earlier error, and was rewritten to demand an explicit re-derivation test. Round 2’s perturber update shifted strategy from arithmetic to *definitional* errors — corrupting which quantity a symbol denotes or what a ratio compares — on the observation that these leave every later line internally consistent, so re-derivation alone cannot surface them. Round 3’s verifier update responded by promoting a “model audit against the problem statement” to equal standing with re-derivation. That edit is the one that fails to transfer.

5.2 TRANSFER TO PROCESSBENCH

Table 2 reports each verifier version on the same 80 held-out items.

Round 2’s edits transfer cleanly: pooled F_1 rises $0.869 \rightarrow 0.897$. Round 3’s do not: $0.897 \rightarrow 0.874$. At the item level the movement is unusually clean in both directions. From v2 to v3, exactly two items change and both change from wrong to right (one MATH, one OlympiadBench), with no regressions. From v3 to v4, exactly two items change and both change from right to wrong, both on OlympiadBench, with no gains.

5.3 ANATOMY OF THE ROUND-3 REGRESSION

The loss is confined to a single cell. Error detection does not move between v3 and v4 ($\text{Acc}_{\text{err}} = 0.85$ for both); OlympiadBench Acc_{cor} falls from 1.00 to 0.80. The v4 policy kept every detection v3 had and added two false alarms on clean chains.

This is the predictable consequence of the two v3→v4 edits, which pushed in the same direction: one gave the definitional audit equal weight with re-derivation, the other asked for more separately-anchored claims. The verifier’s own round report describes the mechanism without knowing it was being measured on it — it flagged chains that “reach a correct final answer through an invalid step. . . following the rule that a chain can be arithmetically fine and still be wrong about the model”. Against the perturber, that rule earned recall: it was written from an episode in which two genuine definitional errors went unclaimed. Against human-annotated chains, ProcessBench’s labels disagree with enough of those judgements to cost more than the rule gains.

This is the outcome the external evaluation exists to detect, and the self-play reward could not have detected it: from inside the game, round 3 looked like a verifier that had closed a real gap. Note the shape of the failure. It is not that skill tuning does not work — round 2’s edits produced a genuine $+0.029$ — but that the round in which the perturber shifted strategy is the round in which the verifier over-fit to that strategy.

Table 3: Published ProcessBench F_1 (%) from Zheng et al. (2024), Table 3, on the full benchmark. Shown for scale only; see the caveats in the text.

Model	GSM8K	MATH	Olympiad.	Omni-MATH	Average
Qwen2.5-Math-7B-PRM800K (PRM)	68.2	62.6	50.7	44.3	56.5
GPT-4o-0806 (critic)	79.2	63.6	51.4	53.5	61.9
QwQ-32B-Preview (critic)	88.0	78.7	57.8	61.3	71.5
o1-mini (critic)	93.2	88.9	87.2	82.4	87.9

5.4 CONTEXT: PUBLISHED PROCESSBENCH BASELINES

Table 3 reproduces reference points from Zheng et al. (2024). These are *not* directly comparable to Table 2 and we do not claim a state-of-the-art result: the published figures use the full 3,400-case benchmark with its natural class balance, our figures use 80 balanced items; the baselines are 2024-era models prompted as single-pass critics, while our verifier is a 2026 frontier model following a tuned multi-step policy inside an agent harness. The table establishes only the scale of the task and the difficulty gradient across subsets.

Two observations survive the caveats. First, the difficulty gradient reported for every model in Zheng et al. (2024) — monotone decline from GSM8K to Omni-MATH — is only weakly present in our runs, where Omni-MATH is the weakest subset (0.800 throughout) but MATH and OlympiadBench are not separated from GSM8K. Second, Omni-MATH is the one subset that no round moved at all, which is consistent with its errors being of a kind the perturber never produces.

5.5 REWARD-IMPLEMENTATION DEFECTS SURFACED BY THE FILTER

The faithfulness filter $\mathcal{A}$ logged 18 candidate scorer defects across the six update reports of three rounds — about ten distinct issues, since the two updaters independently flag the same matcher artifacts — none of which entered a policy. The dominant class is span-alignment brittleness: in three round-2 episodes a correct detection failed to match its ground truth by a one- or two-character margin (a trailing period, a closing bracket) that normalisation does not strip, and in one round-3 episode a claim that correctly diagnosed a false identity scored 0.441 and 0.467 against the two errors it spanned — both just under the $\theta = 0.5$ gate. The aggregate effect is material: round 2’s recorded -0.067 verifier regression is, on the updater’s corrected reading, approximately $+0.058$. We report the uncorrected numbers in Table 1 and treat the discrepancy as a measurement limitation rather than adjusting the results. The methodological point is that the filter converts what would have been silent reward-hacking pressure into an auditable list of bugs.

6 LIMITATIONS

The evaluation is small. Eighty items, ten per class per subset, means each subset accuracy moves in increments of 0.1 and each reported transition is a two-item change; Wilson intervals on the pooled accuracies span roughly ± 0.12 , so neither the $+0.029$ nor the -0.023 is individually significant. We report them because the item-level direction is unanimous in both cases, not because the sample supports a statistical claim. Scaling to the full 3,400 cases is the obvious next step.

We have no measurement of the untuned seed policy. The first evaluation was run after round 1, so `verify-v1` has no ProcessBench score, and we cannot separate the quality of the hand-written seed from the contribution of tuning. Self-play rounds are likewise small (8 episodes), and only one seed and one problem source were used.

Finally, the perturber’s error distribution is not the distribution ProcessBench samples from. Injected misconceptions in Hendrycks MATH solutions are, by construction, localised and of known type; ProcessBench errors arise naturally in model-generated chains and include omissions and invalid framings that our taxonomy does not generate. Transfer across that gap is precisely what we measure, and the round-3 result shows the gap is wide enough to punish a policy that specialises.

7 CONCLUSION

We formulated adversarial self-play over natural-language policies, in which a perturber and a verifier are versioned skills read by an agent, scored by the same programmatic reward an RL implementation would use, and rewritten between rounds by a static updater. Three structural choices — an invariant contract the updater cannot edit, a filter that converts scorer-exploiting wins into bug reports, and enforced context isolation between the players — make the loop auditable in a way that a weight-space implementation of the same game is not: every policy, every edit, and every rejected lesson is a readable artefact tied to the episodes that motivated it.

Our initial results are two rounds of transfer and one round of over-fitting. We regard the second as the more useful finding. Self-play produced a verifier that was measurably better at beating its own opponent and measurably worse at the task, and no quantity available inside the game distinguished the two. Held-out human-annotated evaluation is therefore not a reporting convenience for adversarial skill tuning; it is the only available definition of success.

REFERENCES

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Anthropic. Claude code. <https://claude.com/claude-code>, 2025.
- Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamile Lukošiušė, Amanda Askell, Andy Jones, Anna Chen, et al. Measuring progress on scalable oversight for large language models. *arXiv preprint arXiv:2211.03540*, 2022.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*, 2024.
- Elizabeth Gibney. AI tools are spotting errors in research papers: inside a growing movement. *Nature*, 639, 2025. News feature; doi:10.1038/d41586-025-00648-5.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Bethany Rittle-Johnson, Rebecca Adler, Kelley Durkin, L. Burleigh, Jules King, and Scott Crossley. Detecting math misconceptions: An AI benchmark dataset. In *Proceedings of the Artificial Intelligence in Measurement and Education Conference (AIME-Con), Volume 2: Works in Progress*, pp. 20–24. National Council on Measurement in Education, 2025.
- William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. Self-critiquing models for assisting human evaluators. *arXiv preprint arXiv:2206.05802*, 2022.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shuzheng Si, Haozhe Zhao, Yu Lei, Qingyi Wang, Dingwei Chen, Zhitong Wang, Zhenhailong Wang, Kangyang Luo, Zheng Wang, Gang Chen, Fanchao Qi, Minjia Zhang, and Maosong Sun. From context to skills: Can language models learn from context skillfully? *arXiv preprint arXiv:2604.27660*, 2026.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.
- Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. ProcessBench: Identifying process errors in mathematical reasoning. *arXiv preprint arXiv:2412.06559*, 2024.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations (ICLR)*, 2023.

A APPENDIX: THE EDIT THAT DID NOT TRANSFER

The verifier policy is a numbered list of rules in the tuned segment ϖ_V . Below is rule **V3** as it stood in `verify-v3` (ProcessBench $F_1 = 0.897$) and after the round-3 update produced `verify-v4` ($F_1 = 0.874$). The update was motivated by an episode in which two genuine definitional errors went unclaimed; it is reproduced verbatim except for line wrapping.

`verify-v3.`

V3 — Sweep for the standard misconceptions, then check the model itself. Sign handling across a move between sides; numerator/denominator or dividend/divisor order; common denominators; additive reasoning where the relation is multiplicative; off-by-one in sequence indices; probability outside $[0, 1]$; a step that quietly stops short of what the problem asked. Add the ones that hide best: an inverse operation applied in the wrong direction, and an identity or factorisation whose *form* is invalid while its constants look plausible. Then do a pass that arithmetic cannot do for you — read every definition against the problem statement. ... An error planted in a definition leaves every later line consistent with it, so re-derivation returns nothing; it is invisible except by comparison with the problem.

`verify-v4.`

V3 — Sweep the misconceptions, then audit the model against the problem statement. ... Then do the pass that arithmetic cannot do for you, **and give it the same weight as the re-derivation, because it is where the errors you miss actually live.** Go back to the problem statement and check the solution’s model against it line by line: what each symbol denotes, what a named ratio compares, which set or range a stated condition actually picks out, which quantity is being minimised or maximised, and whether the thing finally reported is the thing that was asked for. A wrong *set membership* or a wrong *objective* leaves every subsequent computation internally flawless, so no amount of re-derivation will surface it — a solution can be arithmetically perfect and still answer the wrong question. ...

The promotion from “also check the model” to “give it the same weight” is the whole regression. Against the round-3 perturber, which had itself just specialised in definitional corruption, the rule recovered exactly the errors the previous version missed. On ProcessBench it produced two false alarms on OlympiadBench chains that are arithmetically sound and whose framing the annotators accepted, and no compensating detections: error accuracy is unchanged at 0.85, correct-chain accuracy on that subset falls from 1.00 to 0.80.

Two properties of this failure are worth noting for the design of such loops. First, it is *legible*: the responsible edit is a single paragraph in a versioned text file, cited to the episode that motivated it, which is not true of the corresponding weight update in an RL implementation. Second, it is *invisible from inside the game* — the same edit raised self-play recall on the round that motivated it, and the faithfulness filter did not catch it because the behaviour is not scorer exploitation at all. It is a correct generalisation from the opponent’s distribution to a policy, and the opponent’s distribution was simply not the world’s.